WorldEpisode: Diagnosing and Correcting Hidden World–Episode Failures in Robot-Learning Datasets

Alba Tellez Fernandez
URDF Studio

WorldEpisode Contributors

Abstract

Robot-learning pipelines can fail while every file still looks valid: a controller may interpret degrees as radians, a replay may treat a command timestamp as an actuation timestamp, or a random train/test split may leak the same reconstructed room into both sides of an evaluation. We show that these are not bookkeeping annoyances but measurable threats to robot-learning validity under scoped, reproducible tests. On a public ArmnetBench LeRobot release, a standard random episode split leaks every tested world lineage and gives a Torch behavioral-cloning probe 0.850 offline imitation success; enforcing a scene-disjoint split drops leakage to zero and the same offline score to 0.000. On a real SO-101 LeRobot trajectory, ignoring the inferred four-frame, 133 ms control delay gives 4.732 degree validation joint RMSE, while timestamp-aware replay reduces it to 1.862 degrees; a tested MuJoCo position-servo replay drops from 3.425 to 1.563 degrees. We introduce *WorldEpisode*, a storage-neutral sidecar contract that makes these silent failures detectable by binding each episode to an immutable world revision through persistent identity, explicit space/time, representation roles, interaction events, and provenance. WorldEpisode does not replace LeRobotDataset, Rerun, NCore, MCAP, OpenUSD, or glTF Gaussian splats; it specifies the invariants those systems must preserve for reproducible conversion, replay, and evaluation. The reference implementation includes active ten-episode public LeRobotDataset-to-WorldEpisode-to-LeRobotDataset batch conversions across two datasets, executable binding artifacts, a semantic validator, independent fixtures, an ACT/Diffusion leakage gate, and a pilot natural-source corpus. It also defines a meta-simulator adapter contract so MuJoCo, Isaac, Genesis, SAPIEN, or future runtimes can report against the same replay invariants. Across seven pilot bindings, native containers preserve 17–39% of a predeclared 23-field semantic projection, while WorldEpisode sidecars recover that projection for dataset/log/world bindings. The validator detects all 14 injected fault classes and all hand-authored independent fixture failures; the pilot natural-source corpus records nine cases across three public LeRobot-format datasets. The central result is simple: bad data plumbing can make robot-learning results look better than they are, and explicit world–episode semantics make those failures auditable before deployment.

1 Introduction

Imagine training a behavioral-cloning policy for three weeks on a public robot dataset. The loss curves are smooth, the simulator reports an 85% success rate, and every Parquet, video, and scene file passes its local loader. Then the policy is deployed on the real arm and immediately misses the object. The failure is not a new neural architecture problem. The dataset silently encoded joint values in a unit the controller did not declare, the replay script treated command timestamps as motor-effective timestamps, and the random split let the policy see the same reconstructed room in both training and evaluation. Nothing was corrupt, but the experiment was scientifically broken.

This is the failure mode WorldEpisode targets. Modern robot-learning pipelines no longer move only arrays from disk to a policy. Recorded episodes become training datasets; reconstructed rooms become simulator assets; Gaussian splats, meshes, collision proxies, and logs move through different tools; and counterfactual worlds become augmentation. A camera transform with the wrong direction, a millisecond command delay, or a train/test split that shares the same world lineage can change the conclusion without changing file validity.

Existing formats solve important pieces of this chain. The LeRobot library vertically integrates robot learning from motor interfaces through dataset collection, storage, streaming, policies, and deployment [3]. The current LeRobotDataset v3 layout stores high-frequency states, actions, and timestamps in Parquet, visual streams in MP4 shards, and episode indexing in metadata rather than filenames [6]. Rerun provides a physical-data layer and storage target for multimodal recordings. NCore documents explicit pose graphs and sensor conventions. OpenUSD, SimReady, glTF, and simulator-native formats describe composed worlds and assets. Gaussian splats are also moving into standard asset containers through glTF's `KHR_gaussian_splatting` extension [14, 15]. Recent real-to-sim systems such as RoboSnap and DROID-Sim show that Gaussian-splat environments can support trajectory replay, synthetic data generation, and policy evaluation at dataset scale [13]. That progress makes the missing contract more urgent rather than less. A layered real-to-sim scene can be visually faithful while still dropping the collision role of an object, and a replayable trajectory can still be wrong if the policy action vector is interpreted under a different controller contract at deployment time.

The missing piece is not another monolithic file. A LeRobot table can store actions without knowing which immutable world revision they came from; a USD scene can store assets without knowing the policy action semantics that generated a trajectory; a ROS or MCAP log can store messages without knowing whether evaluation splits leak the same physical room. WorldEpisode fills that gap as a sidecar contract: it binds each episode to the world in which it happened and records the assumptions required to convert, replay, and evaluate it.

WorldEpisode is:

- **storage-neutral**: serializable through LeRobotDataset v3, Rerun, NCore, MCAP, or a reference package;
- **representation-neutral**: compatible with splats, meshes, NeRFs, point clouds, and future assets;
- **runtime-neutral**: mappable to Isaac Sim, MuJoCo, SAPIEN, Genesis, and other simulators;
- **loss-explicit**: conversions may be lossy, but never silently lossy.

The paper's core claim is therefore not a new monolithic file format. It is an executable interchange profile for world–episode semantics: identity, frames, clocks, action contracts, representation roles, events, provenance, uncertainty, and lineage-safe evaluation splits. The current artifact includes a JSON Schema, semantic validator, round-trip binding artifacts, an active public LeRobotDataset v3 conversion experiment, hand-authored independent fixtures, and controlled experiments for replay drift, leakage, and counterfactual augmentation. The empirical claims in this draft are intentionally bounded: they demonstrate executable failure mechanisms and auditable conversion semantics, not a completed multi-lab benchmark or real-robot deployment study.

2 Problem Statement

WorldEpisode starts from a practical question: what must be known before an episode can be safely converted, replayed, augmented, or used for evaluation? A valid episode record must answer five questions. A production dataset must also answer how those records are named, sharded, indexed, resolved, and versioned without turning storage layout into the semantic API.

Which world did this happen in? Each episode references one immutable, content-addressed world revision plus an ordered list of episode-specific deltas. Changing geometry, metric scale, calibration, collision proxies, physical parameters, entity decomposition, object identity, or task-relevant fixtures creates a new world revision. This makes it detectable when two datasets silently use the same filename for different physical worlds, or different filenames for the same world.

What did the robot see and touch? Objects, robot links, sensors, fixtures, support surfaces, semantic regions, and generated assets must have persistent identifiers. A mug in a segmentation mask, a Gaussian group, a collision mesh, a simulator actor, a language annotation, and a contact event should resolve to the same entity when they refer to the same physical object.

Where and when did it happen? Spatial values declare their frames, units, transform direction, validity interval, and calibration revision. Streams declare clock domains, timestamps, synchronization mappings, and timing error where available. Actions declare command time and effective motor time rather than assuming those are the same instant.

What was preserved or lost during conversion? WorldEpisode reuses existing standards rather than replacing them. USD [1] carries composed simulation worlds, glTF carries Gaussian appearance assets [14], NCore carries capture and pose-graph conventions [10], Rerun carries physical-data recordings with column chunks [12], and ROS/MCAP carries logs. A converter may externalize or approximate fields, but it must emit a machine-readable loss report instead of silently discarding semantics.

Can the evaluation split leak? Episode-level random splits are not enough when multiple episodes share a reconstructed room, physical object, source capture, calibration, or generated asset lineage. A WorldEpisode split manifest can enforce disjointness by world lineage, physical entity, source capture, reconstruction run, or generated asset family.

Can this scale past one dataset folder? Large robot corpora require a dataset manifest above the episode records. The manifest declares namespaces, resolver priority, shard catalogs, materialized indexes, split catalogs, and append-only versions. This lets a reader locate all SO-101 pick-place traces for a world-disjoint split by index lookup rather than by crawling storage paths.

The initial profile is intentionally narrow: rigid tabletop manipulation with fixed-base single- or dual-arm robots. That bounded domain keeps conformance executable rather than aspirational; later profiles can add humanoids, locomotion, deformables, fluids, and multi-agent worlds.

3 WorldEpisode Architecture

WorldEpisode is implemented as a sidecar blueprint around existing formats. It does not ask a LeRobot dataset to become a USD scene, or a USD scene to become a policy-training table. Instead, it records the relationships those containers need in order to agree about the same episode: the world revision, persistent entities, space/time conventions, action contract, asset roles, interaction events, provenance, and split lineage.

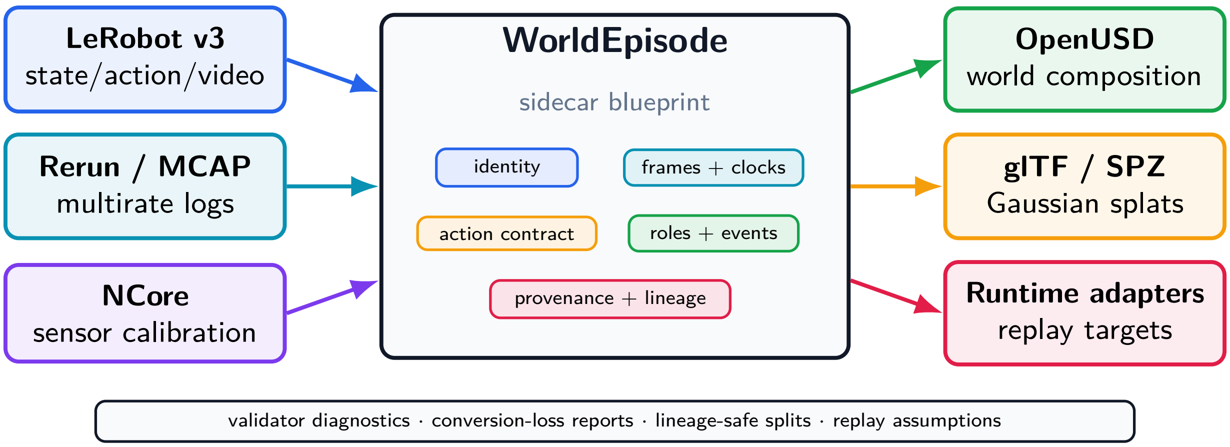

Figure 1: WorldEpisode is the semantic bridge between robot-learning containers, reconstructed world assets, and simulator targets. It preserves the world–episode contract without replacing the storage formats on either side.

The sidecar is not a mandate to package a global robot corpus as millions of small folders. The single-episode record is the semantic unit. Production datasets add a manifest layer: globally scoped namespaces, URI resolvers, shard catalogs, materialized indexes, split manifests, and append-only release snapshots. Opening such a corpus starts from the manifest and its indexes; it does not require recursively scanning an object store or loading every episode.

Figure 1 shows the intended data flow. Dataset and log containers plug in on the left; world assets and replay targets plug in on the right; the WorldEpisode sidecar holds the semantics that would otherwise be scattered across undocumented loader assumptions.

This architecture gives the paper its central rule: keep native containers for the data they are good at storing, but make the cross-container assumptions explicit, checkable, and loss-aware.

3.1 WorldEpisode as a Meta-Simulator Contract

WorldEpisode is not tied to a single rendering or physics engine. It operates as a meta-simulator contract: a runtime becomes a trusted target by implementing an adapter that preserves the same semantic invariants and emits replay evidence. This keeps the integration direction clean. Datasets are not rewritten around simulator-native assumptions; simulators compile their scene loading, action interfaces, and replay reports against the WorldEpisode sidecar.

The adapter contract has three layers. First, the *invariant interface* ingests immutable world revisions, frame and clock graphs, entity identity, representation roles, action channels, asset digests, provenance, and quality records. Second, *asynchronous schema extension* allows an engine to register cloth, deformable, fluid, articulation, material, or procedural-scene extensions without weakening the core episode record. Third, *deterministic replay accountability* records runtime identity, version, solver, timestep, actuator parameters, initialization state, command/effective timing, latency, tolerance envelopes, and measured drift.

The current implementation reflects this boundary. MuJoCo has one tested minimal replay adapter; the Isaac mapping is emitted but untested; Genesis and SAPIEN are explicit adapter-required targets. The claim is therefore not simulator-independent physics. The claim is runtime-neutral accountability: every simulator reports against the same world–episode contract, so cross-runtime drift becomes measurable rather than hidden inside loader conventions.

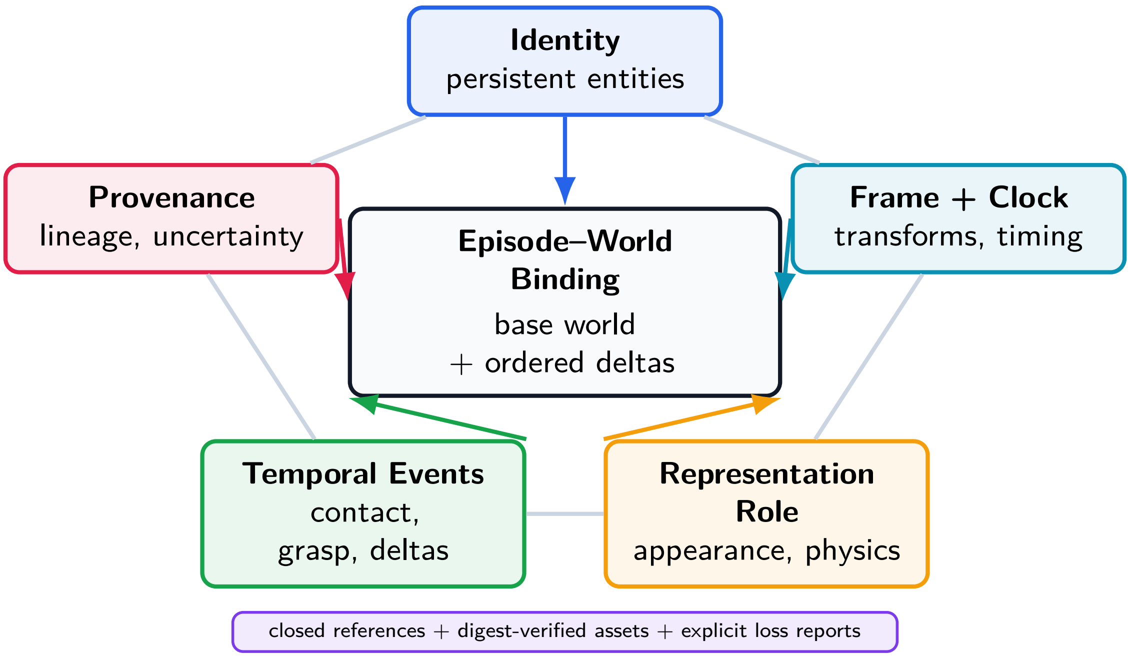

Figure 2: The five-graph model. WorldEpisode makes identity, frames/clocks, representation roles, temporal events, and provenance mutually referential so conversion and replay errors become detectable instead of hidden.

4 Five Questions, Five Graphs

Figure 2 shows the five coupled graphs that make an episode replayable and auditable rather than just stored. Each graph answers a question that appears during debugging: what object is this, where and when is it, what is the asset safe to do, what physically happened, and where did this data come from?

4.1 What object am I looking at?

Every physical or logical entity receives a persistent identifier: robot, link, gripper, sensor, rigid object, articulated object, fixture, support surface, region, human, light, or semantic group. The same `entity_id` must be used by segmentation masks, bounding boxes, Gaussian groups, render meshes, collision bodies, simulator actors, language references, contact events, object trajectories, grasp events, and attachment events. The real-world fix is simple: if a language command says "pick up the mug," the perception mask, the simulator object, and the collision proxy must identify the same mug.

4.2 Where is it, and exactly when?

Every spatial value declares source frame, target frame, units, handedness, up axis, transform direction, quaternion convention, validity interval, calibration revision, and uncertainty where available. Every stream declares a clock domain, device timestamp, host-receive timestamp where available, exposure interval for cameras, effective command time for actions, synchronization mapping, and estimated offset/drift/error. NCore's explicit pose-graph convention demonstrates the kind of rigor WorldEpisode requires across the whole episode [10]. This graph is what turns a suspicious replay from "the policy is bad" into a diagnosable offset, transform direction, or unit error.

4.3 What is this asset safe to be used for?

One physical entity can have many representations: Gaussian splat appearance, textured render mesh, metric surface mesh, depth point cloud, collision proxy, mass/inertia record, semantic affordance, or learned embedding. Each representation declares its role and an asset descriptor: URI, media type, SHA-256 digest, optional mirrors, optional embedded payload metadata, coordinate frame, units, validity interval, derivation, uncertainty, and license. The URI may be relative, HTTP(S), Hugging Face, object storage, OCI, IPFS, or another registered resolver. Portability comes from deterministic resolution and digest verification, not from requiring every asset to live in one folder. The role field is essential: a splat may be valid for appearance but invalid for collision, while a simplified proxy may be valid for collision but unsuitable for visual training.

4.4 Did the robot actually grab the object?

Episodes do not store only poses. They record interaction events: contact begin/end, grasp establish/release, attachment create/remove, support, containment, articulation state changes, spawn/despawn, topology changes, simulator reset, intervention, subgoal completion, and failure detection. Each event references entities, time, confidence, source, and optionally evidence. This turns raw coordinate streams into physical milestones such as "contact established," "object dropped," or "intervention occurred."

4.5 Where did this data come from?

Every derived asset identifies source episodes, source sensors and intervals, reconstruction algorithm, software version, model/checkpoint hash, configuration, calibration revision, manual edits, source and output hashes, creator, license, and quality metrics. WorldEpisode maps to existing provenance and dataset metadata vocabularies rather than inventing an incompatible publication vocabulary. This graph lets a split avoid putting the same reconstructed room, capture session, or generated asset family on both sides of evaluation.

5 Reference Implementation

5.1 Complete Action Semantics

An action vector is not portable without its operational contract. Each action channel must declare actuator, control mode, parameterization, reference frame, units, absolute/delta/velocity semantics, limits, command rate, command timestamp, effective timestamp, latency model, interpolation, normalization, gripper semantics, and missing-value policy. A vector such as $[0.01, 0, 0, 0, 0, 0, 1]$ is otherwise ambiguous: it could be a tool-frame delta, base-frame delta, velocity command, normalized policy output, or delayed target increment.

5.2 Immutable World Revisions

Each episode references one base world revision plus ordered episode deltas. A new revision is required when world geometry, metric scale, calibration, collision proxies, physical parameters, entity decomposition, object identity, or task-relevant fixtures change. Content hashes allow two datasets to prove that they refer to the same world without relying on filenames.

5.3 Loss-Aware Conversion

WorldEpisode supports many bindings, but it does not promise impossible universal losslessness. Every converter emits a machine-readable report listing preserved, externalized, approximated, discarded, and warning fields. The standard permits lossy conversion; it forbids silent lossy conversion.

5.4 Implementation Artifact

We implement the contract as four executable components: a JSON Schema for structural validity, a machine-readable conformance requirement file, an installable semantic validator, and a round-trip experiment runner. The validator maps each diagnostic to a stable requirement identifier such as `TIME.001`, `FRAME.002`, or `ASSET.002`. The experiment runner takes a WorldEpisode package, exports binding-native artifacts plus sidecars, reimports them, compares a semantic projection, injects controlled faults, and writes reproducible result artifacts. URDF Studio remains the initial implementation root, but the contract is represented independently in the public schema and validator.

The validator is packaged as `worldepisode`. A blocking preflight can be run from the shell with `worldepisode preflight <dataset-or-manifest>` or from a training script with a single Python call such as `preflight_lerobot(dataset.root).raise_if_failed()`. Native LeRobot folders and Rerun `.rrd` recordings are recognized directly; without a WorldEpisode manifest or sidecar, they fail closed on missing world revision, entity identity, representation role, action timing, frame/clock, split-lineage, and conversion-loss controls. This makes the semantic audit a pre-training check rather than a separate post-hoc documentation exercise.

The repository also includes an active LeRobotDataset v3 converter. It downloads bounded Parquet and metadata shards from the pinned public `lerobot/svla_so101_pickplace` SO-101 dataset, constructs a schema-valid WorldEpisode manifest and sidecar, exports a small LeRobot-Dataset v3 package, reads that package back, and asserts equality of source-native tensors and timestamp records. Source fields that LeRobot does not declare, such as camera extrinsics and controller latency, are not guessed; they are emitted in the conversion report as explicitly source-absent. The split-leakage tool similarly downloads bounded ArmnetBench LeRobot shards, writes `world_lineage` and split-manifest artifacts, and trains an executable BC baseline over the real LeRobot tensors. The control-replay tool reads the exported LeRobot trajectory, estimates the effective command delay from action/state alignment, writes the WorldEpisode action contract, executes the tested MuJoCo adapter, and emits an Isaac adapter mapping that is ready for an Isaac environment but explicitly marked untested in the committed report.

The repository also emits a meta-simulator adapter report. It defines three compliance layers: the invariant interface, asynchronous schema extension, and deterministic replay accountability. The report records MuJoCo as the current tested minimal adapter, Isaac as adapter-ready but untested, and Genesis/SAPIEN as adapter-required targets. This artifact is deliberately an adapter contract rather than a claim of equal runtime coverage.

6 Failure Case Studies

The experiments are written as failure investigations rather than as isolated unit tests. Each one starts from a failure mode that can make a robot-learning result look better, cleaner, or more reproducible than it really is.

Case 1: the split that faked success. The most dangerous failure is not a crash; it is a benchmark that looks successful for the wrong reason. We audit ArmnetBench v0.1's single-arm SO-101 LeRobot release [2] by deriving eight WorldEpisode-style `world_lineage` hashes, one for each task-scene and camera-layout group. A standard random episode split trains on 320 episodes, tests on 80, and leaks all test scene lineages into training. The same Torch MLP state-action behavioral-cloning probe then reaches 0.850 offline imitation success under a fixed normalized-RMSE tolerance of 0.25.

When the split is made scene-disjoint, the apparent success disappears. The held-out eye-drop scene lineages have zero overlap with training, the test set contains 100 episodes, and offline BC success drops to 0.000. Mean episode normalized RMSE rises from 0.183 to 0.363, a 1.98× increase. This is an offline action-imitation benchmark, not a physical rollout claim or a claim about state-of-the-art LeRobot policies. Its purpose is narrower and diagnostic: it exposes that a conventional split can reward memorizing a background world rather than learning a transferable manipulation behavior. The committed split manifest makes the same leakage audit repeatable with ACT, Diffusion Policy, or another LeRobot-native policy.

To keep that stronger claim from being implied before it is measured, the repository also emits an ACT/Diffusion gate from the same split manifest. The gate writes train/test episode allowlists, four LeRobot-native training jobs (`act` and `diffusion` on random and scene-disjoint splits), required offline action-evaluation reports, and a high-fidelity simulator or physical rollout contract. The gate is intentionally open in this draft; no ACT, Diffusion Policy, IsaacLab, or hardware success number is claimed.

Figure 3 summarizes the measured changes for three experiment families where world–episode semantics alter the outcome.

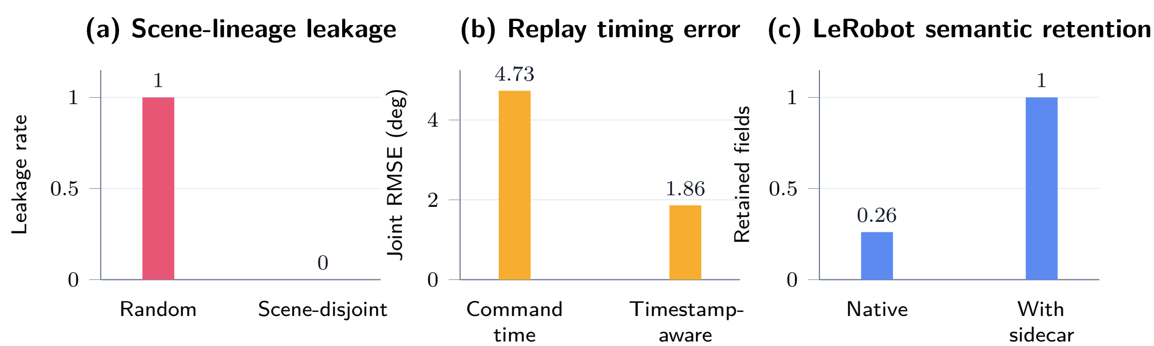

Figure 3: Quantitative summary of the three measured effects used in the evaluation. Lower is better for leakage and replay RMSE; higher is better for semantic retention.

Case 2: the converter that refuses to invent missing physics. To test an actual LeRobot-Dataset rather than only a capability model, we implement LeRobotDataset v3 → WorldEpisode → LeRobotDataset v3 on five-episode batches from two pinned public datasets: `lerobot/svla_so101_pickplace` [8] and `lerobot/pusht` [7]. The converter downloads the real Parquet metadata and data shards, builds a schema-valid WorldEpisode manifest and sidecar, exports a compact LeRobotDataset v3 package, reads it back, and asserts exact equality.

This is a two-dataset batch audit for active conversion, not a coverage claim for all LeRobot datasets. Across ten episodes, the round trip preserves 1,935 rows of source-native action tensors, 1,935 state rows, sample timestamps, frame indices, episode indices, global sample indices, task indices, and camera video timestamp ranges with maximum absolute error 0.0. It also preserves the derived physical-frame records through the sidecar. More importantly, it does not fake missing semantics: camera extrinsics, robot/world calibration, action units, and controller latency are reported as source-absent rather than guessed. This is the difference between a convenient conversion and an auditable one.

Case 3: the replay that drifted because time was underspecified. LeRobot's control stack can decouple policy inference from the fixed-rate low-level controller: commands may be queued, chunked, interpolated, or consumed after policy output [3]. On the public SO-101 trajectory, WorldEpisode records policy-output, enqueue, queue-consume, and motor-receive timestamp semantics, plus zero-order-hold interpolation and a constant-frame latency model.

Estimating the effective delay on the calibration prefix gives four 30 Hz frames, or 133 ms. On the validation suffix, naive command-time alignment has joint RMSE 4.732 degrees. Timestamp-aware alignment has RMSE 1.862 degrees, a 2.54× improvement. In a tested MuJoCo six-joint position-servo replay, naive command-time replay has RMSE 3.425 degrees and timestamp-aware replay has 1.563 degrees, a 2.19× improvement. The Isaac adapter is emitted as a ready mapping from the same contract, but is not tested in this environment. The failure avoided here is a common one: blaming a simulator or controller when the dataset never said when the command actually took effect.

Case 4: real-to-sim contract drift. Recent Gaussian/OpenUSD real-to-sim systems correctly separate visual reconstruction from interactive physical assets. WorldEpisode targets the contract between those layers and the source robot episode. We therefore add two deterministic ablations. In the action-contract ablation, a policy succeeds in a simulator that interprets its output as absolute joint-radian targets, but the same vector fails under a deployment proxy whose controller expects delta-degree commands. In the representation-role ablation, an appearance-only Gaussian export drops the collision proxy; the straight-line policy succeeds in simulation and collides with the real foreground geometry under the deployment proxy. In both ablations the drifted contract has 2/2 simulated successes and 0/2 deployment successes, while the WorldEpisode contract recovers 2/2 deployment successes. This is a controlled proxy, not a RoboSnap or hardware rerun, but it shows why visual reconstruction is not enough without action contracts and representation roles.

Case 5: the validator that catches physically impossible packages. We inject 14 faults into the baseline package: missing clock domains, incomplete cross-clock mappings, unknown frames, missing transform intervals, unknown event entities, missing representation roles, digest errors, action-contract omissions, non-content-addressed worlds, missing delta sequences, missing provenance, and missing quality records. The validator detects all 14 expected requirement failures with 1.000 recall and 0.933 precision. Two hand-authored independent fixtures also trigger the expected diagnostics. The single false-positive requirement is useful rather than harmful: removing an action reference frame triggers both `ACTION.001` and the downstream `FRAME.001` reference check.

The same suite includes a simple unit sanity check from `lerobot/svla_so101_pickplace`. The first mirrored SO-101 frame has six joint values with maximum magnitude 98.924 in degrees and

1.727 after radian conversion. Treating those degree values as radians would violate the action/state unit contract even though the tensor shape is valid.

To move beyond synthetic mutations, the experiment runner also writes a pilot natural-source corpus from committed public LeRobot artifacts. Across `lerobot/svla_so101_pickplace`, `lerobot/pusht`, and `armnet/armnetbench_v01_lerobot_so101`, it records nine observed cases: source-absent camera extrinsics, robot/world calibration transforms, action units, controller-latency models, and a random episode split with measured scene-lineage leakage. These are not claimed as maintainer-confirmed dataset bugs; they are naturally occurring source omissions or evaluation risks that a WorldEpisode audit must make explicit.

The same validator is exposed as a pre-training command rather than only as an experiment script. The preflight regression covers four cases: a valid WorldEpisode manifest passes, an invalid WorldEpisode fixture fails, a native LeRobot v3 folder without a WorldEpisode sidecar fails closed, and a Rerun `.rrd` recording without a sidecar fails closed. This is the intended usage pattern for labs: run one blocking line before launching a multi-day policy training job.

Case 6: the famous benchmarks that still need call-out audits. The ArmnetBench result is the measured leakage case. To scale the same diagnostic, we add a source-level call-out audit over five famous public robot-learning benchmarks: Open X-Embodiment/RT-X [11], DROID [5], BridgeData V2 [16], LIBERO [9], and CALVIN [4]. The audit asks whether public metadata exposes world or scene lineage splits, content-addressed world revisions, persistent entity identity, action timing and latency contracts, frame/clock graphs, and replay or conversion-loss reports. All five benchmarks have at least one high-severity open control; the three real-robot datasets have five each in the current source-level audit. This is a call-out target list, not a claim that their published scores are inflated. The stronger claim requires converting each benchmark into WorldEpisode and rerunning a published policy protocol under lineage-disjoint splits or timestamp-aware replay.

Case 7: augmentation that uses the world rather than a side file. We evaluate a shifted-camera/object-displacement regression task. Observations-only nearest-neighbor control reaches the proxy success threshold on 0.292 of shifted test cases. Adding unstructured 3D side information increases proxy success to 0.342. WorldEpisode-style world-frame identity and counterfactual camera augmentation increase proxy success to 0.592 over 120 deterministic test cases.

Binding retention. The same executable artifacts measure how much of the semantic contract is preserved by native containers alone. Each binding writes a native payload and a WorldEpisode sidecar, then the experiment reimports both and checks exact equality against a predeclared 23-field semantic projection: episode identity/task outcome, world revision and asset descriptor, embodiment identity and URDF asset, frames/transforms, clocks/mappings, entity identity/roles/assets, action control and timing contracts, trace binding and asset descriptor, events, ordered deltas, provenance, quality, split lineage, and replay assumptions. This is an executable pilot projection, not a universal score of each format's value. Table 1 shows that common containers preserve 17–39% of this projection natively, while the sidecar recovers all fields for the tested dataset/log/world bindings. A Gaussian asset binding alone preserves 17% natively and 30% with limited sidecar metadata, confirming that an asset format is not an episode–world contract.

Finally, we generate episodes with world/entity lineage labels and evaluate a memorization policy. A random episode split leaks every tested world/entity pair and obtains 1.000 accuracy. World-disjoint and entity-disjoint splits have zero measured leakage and reduce memorization

Table 1: Semantic field retention in the pilot binding model.

Binding	Native	With WorldEpisode
LeRobot v3	0.261	1.000
Rerun RRD	0.348	1.000
NCore	0.304	1.000
MCAP/ROS2	0.261	1.000
OpenUSD/SimReady	0.391	1.000
glTF Gaussian asset	0.174	0.304

accuracy to 0.500 and 0.562 respectively. This pilot demonstrates why split manifests need lineage constraints beyond the public ArmnetBench audit above.

7 Limitations and Scope

The current evidence is deliberately scoped. It is meant to expose hidden failure mechanisms, not to claim full robot-learning generality.

Table 2 states the boundary of each empirical claim. This is important because the paper argues for a semantic contract, not for a finished benchmark suite.

Table 2: What the current evidence supports, and what it does not yet support.

Claim area	Current evidence	Boundary
Leakage	Public ArmnetBench LeRobot audit with 400 teleoperated reference episodes, an executable Torch BC probe, and an ACT/Diffusion gate harness	Stronger policy jobs are prepared but not run; no real-robot or high-fidelity rollout result yet
Conversion	Two pinned public LeRobotDataset v3 five-episode batch round trips with exact tensor, index, and timestamp equality	Two datasets; broader LeRobot coverage remains future work
Replay timing	Real SO-101 trajectory alignment and tested MuJoCo position-servo replay	One trace and one MuJoCo adapter; Isaac mapping is emitted but untested
Meta-simulator neutrality	Adapter contract matrix over MuJoCo, Isaac Sim, Genesis, and SAPIEN	One tested minimal MuJoCo adapter; Isaac ready but untested; Genesis/SAPIEN adapters not implemented
Validation	Fourteen injected requirement faults, two independent hand-authored fixtures, and a pilot natural-source corpus over three public datasets	Natural corpus is below the five-dataset gate and has no maintainer feedback yet
Binding retention	Predeclared 23-field semantic projection checked by executable artifacts	Pilot projection; not a universal score of each storage format
Famous benchmark call-out	Source-level audit over Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN	Missing public controls are not measured score inflation; targeted conversions and reruns are still required
Real-to-sim drift	Controlled action-contract and representation-role ablations where drifted contracts succeed in simulation and fail under deployment proxies	Deterministic proxy; not a RoboSnap/DROID-Sim rerun or physical hardware deployment
Adoption	Public schema, installable local preflight package, validator, fixtures, and governance files	No PyPI release, upstream LeRobot/Rerun integration, independent implementation, or external dataset release yet
Scale architecture	Dataset manifest schema, scalable example, and production requirements for shards, indexes, resolvers, and append-only versions	No benchmark yet for billion-episode catalog latency, distributed cache behavior, or federation across institutions

Manipulation scope. The current profile targets rigid tabletop manipulation with fixed-base single- or dual-arm robots. Humanoids, locomotion, deformables, fluids, and multi-agent environments require later profiles and new conformance fixtures.

Offline leakage result. The ArmnetBench experiment is an offline behavioral-cloning audit, not a physical rollout. Its value is that it demonstrates how random episode splits can leak world lineage and inflate evaluation, not that it measures real-robot success directly. The current baseline is an executable Torch MLP probe over LeRobot tensors; ACT and Diffusion Policy should be run on the committed split manifest before claiming impact on common LeRobot training recipes. The repository now emits those ACT/Diffusion jobs and rollout requirements, but no such policy result is claimed here.

Replay is not bit-identical physics. WorldEpisode records simulator assumptions, timing, interpolation, and tolerances; it does not promise simulator-independent behavior. The reported replay result is a MuJoCo position-servo test under declared assumptions. The Isaac mapping is emitted and ready, but intentionally untested in this environment.

Meta-simulator evidence. The meta-simulator section defines what a compliant runtime adapter must preserve. It does not certify that MuJoCo, Isaac, Genesis, SAPIEN, or any other simulator has equivalent physics. Current evidence contains one tested minimal MuJoCo adapter, one untested Isaac mapping, and explicit adapter-required entries for Genesis and SAPIEN.

Reference implementation maturity. The converter, validator, and reports are executable, but independent implementations are still needed before the project should be treated as a mature interoperability standard. The next empirical step is to run the same protocol on larger multi-camera manipulation datasets and externally maintained adapters.

Preflight distribution. The repository packages a local `worldepisode` preflight CLI and Python API, but it is not yet a released PyPI package and has not been merged into upstream LeRobot or Rerun examples. The current claim is friction reduction in the reference implementation, not ecosystem default status.

Dataset-scale performance. The specification separates the core episode record from a dataset-scale manifest with namespaces, shards, indexes, resolvers, and append-only snapshots. That design prevents the standard from being defined around a local folder layout, but it is not yet a measured distributed-systems result. A production paper should benchmark catalog-open latency, partition pruning, asset cache hit rates, and validation throughput on much larger corpora.

Famous benchmark call-out scope. The audit over Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN is a source-level metadata audit. It identifies where public materials do not expose the controls needed to test world-lineage leakage, action latency, or replay loss. It does not prove any published benchmark score is inflated. That claim requires a benchmark-specific WorldEpisode conversion and a measured policy rerun.

Real-to-sim drift scope. The contract-drift experiment isolates two real-to-sim failure mechanisms: action-interface drift and missing representation roles. It is intentionally deterministic and does not use a reconstructed RoboSnap/DROID-Sim scene or a physical robot. The stronger result is to bind a public reconstructed scene, replay the same checks, and measure simulator/deployment divergence under declared tolerances.

Natural failure corpus. The injected conformance faults prove that requirements are executable and diagnosable. The current pilot natural-source corpus adds nine cases from three public LeRobot-format datasets, but it still does not prove prevalence in the wild. A stronger version of this paper should extend the audit to at least five public datasets, report how often each requirement fails, and record whether dataset maintainers agree with representative diagnostics.

8 Discussion

Why this is not only infrastructure. The experiments show three ways a result can be wrong while the files are valid: a split can leak world lineage, replay can drift because control timing is underspecified, and conversion can discard physical semantics that the target container has no native place to store. WorldEpisode is useful only because these failures are observable and testable. The sidecar is the mechanism; the scientific claim is that explicit world–episode semantics change what a robotics result means. The contribution is therefore a set of falsifiable invariants: persistent entity identity must be consistent across observations and assets, action timestamps must distinguish command and effective time, representation roles must separate appearance from physics, and evaluation splits must be able to exclude shared world lineage. A container adapter either preserves those invariants, externalizes them with declared loss, or fails conformance.

Not another monolithic binary. WorldEpisode has a small semantic core and multiple bindings. LeRobotDataset v3 is the policy-training view; Rerun is the multirate physical-data view; NCore is the sensor/reconstruction capture view; MCAP/ROS is the raw log view; OpenUSD/SimReady is the composed simulation-world view; glTF or SPZ is the Gaussian asset view; and a reference directory is the conformance profile.

Real-to-sim as the proving ground. Gaussian-splat and OpenUSD real-to-sim pipelines are not competitors to WorldEpisode. They are the systems that most need an episode contract. A reconstructed room can be excellent visual training data and still be unsafe for policy evaluation if foreground collision roles are missing, if the same world lineage leaks across splits, or if the replayed policy output is interpreted under the wrong hardware action contract. WorldEpisode should be judged by whether it keeps those pipelines auditable as they scale, not by whether it serializes splats better than asset formats designed for that job.

Simulators as accountable adapters. The same principle applies to physics and rendering engines. WorldEpisode should not become a MuJoCo wrapper, an Isaac wrapper, or a Genesis wrapper. It should define the contract those adapters must satisfy: ingest the invariant interface, declare extensions, and report replay assumptions and drift. This is a stronger runtime-neutral stance than claiming universal behavior. It lets a simulator team plug in by implementing the adapter, while the validator remains the common judge of whether the exported dataset is physically and temporally auditable.

Lineage-safe evaluation. Episode-level random splits can leak the same reconstructed room, physical object, Gaussian asset, source video, world revision, or generated asset lineage across training and evaluation. WorldEpisode supports split constraints such as `world_lineage`, `physical_entity`, `source_capture`, `reconstruction_run`, and `generated_asset_family`.

Call-outs without overclaiming. The practical target is to make famous benchmarks auditable, not to accuse them without a rerun. When public metadata cannot answer whether train and test share a scene lineage, asset family, source capture, action timing convention, or simulator assumption, WorldEpisode should mark the published score as unaudited with respect to that control. It should mark a score as inflated only after the same policy protocol is rerun under the corrected split or timing contract.

Standards alignment. The project aligns with standards bodies rather than antagonizing them. Static 3D robot-map exchange, humanoid dataset lifecycle, OpenLABEL annotation, glTF Gaussian splats, USD composition, and NCore capture conventions are all useful neighbors. WorldEpisode is an executable profile and bridge across those layers.

9 Conclusion

Robot-learning failures often arrive disguised as clean data: valid tensors, valid videos, valid scene files, and invalid assumptions. WorldEpisode makes those assumptions explicit. It binds an episode to an immutable world revision, gives persistent identity to physical entities, declares space/time/action semantics, assigns roles to assets, records events and provenance, and reports conversion loss instead of hiding it. In the reference experiments, those requirements expose an 85% offline random-split result that collapses to 0% under scene-disjoint evaluation, reduce LeRobot replay drift on a real SO-101 trajectory by modeling a four-frame action delay, and catch controlled physically incoherent packages before they are used as trusted artifacts. They also show how a real-to-sim pipeline can report simulated success while failing under deployment proxies when action contracts or representation roles drift. The same validator is exposed as a one-line preflight command for WorldEpisode, LeRobot, and Rerun artifacts, and the simulator pathway is defined as an adapter compliance contract rather than a dependency on one runtime. These are scoped results, not a completed standardization claim. The new call-out audit identifies Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN as priority targets for this protocol, but it does not claim their scores are inflated without reruns. Gaussian splats remain a high-value appearance profile, but the durable contribution is the world–episode contract around them. The next step is to scale the same protocol to larger multi-robot datasets, stronger LeRobot-native policies, real or high-fidelity rollout evaluation, natural failure corpora, and independently maintained bindings.

References

[1] Alliance for OpenUSD. OpenUSD: Universal scene description. https://openusd.org/, 2026. Accessed 2026-07-13.

[2] Armnet. ArmnetBench v0.1: LeRobot single-arm SO-101 release. https://huggingface.co/datasets/armnet/armnetbench_v01_lerobot_so101, 2026. Pinned revision 2e5e89aee0e7f081078d9d6ab3b279fc4b83ea84; accessed 2026-07-13.

[3] Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, Mustafa Shukor, Jess Moss, Alexander Soare, Dana Aubakirova, Quentin Lhoest, Quentin Gallouedec, and Thomas Wolf. LeRobot: An open-source library for end-to-end robot learning. In *International Conference on Learning Representations*, 2026. arXiv:2602.22818.

[4] CALVIN Authors. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. https://calvin.cs.uni-freiburg.de/, 2021. Accessed 2026-07-13.

[5] DROID Dataset Authors. DROID: A large-scale in-the-wild robot manipulation dataset. https://droid-dataset.github.io/, 2024. Accessed 2026-07-13.

[6] Hugging Face. LeRobotDataset v3.0. https://huggingface.co/docs/lerobot/lerobot-dataset-v3, 2026. Accessed 2026-07-13.

[7] LeRobot. pusht: A public LeRobotDataset v3 dataset. https://huggingface.co/datasets/lerobot/pusht, 2025. Pinned revision 7628202a2180972f291ba1bc6723834921e72c19; accessed 2026-07-13.

[8] LeRobot. svla_so101_pickplace: A public LeRobotDataset v3 dataset. https://huggingface.co/datasets/lerobot/svla_so101_pickplace, 2025. Pinned revision f641879e22172be7e8161d5e6c1503c2d2feb657; accessed 2026-07-13.

[9] LIBERO Authors. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. https://libero-project.github.io/main.html, 2023. Accessed 2026-07-13.

[10] NVIDIA. NCore Specification: Data Conventions. https://nvidia.github.io/ncore/data/conventions, 2026. Accessed 2026-07-13.

[11] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and rt-x models. https://robotics-transformer-x.github.io/, 2023. Accessed 2026-07-13.

[12] Rerun. A new data layer for robot learning. https://rerun.io/blog/data-layer-for-robot-learning, 2026. Accessed 2026-07-13.

[13] RoboSnap Authors. RoboSnap: One-Shot Real-to-Sim Scene Generation for Generalizable Robot Learning and Evaluation. https://arxiv.org/html/2607.06699v1, 2026. Accessed 2026-07-13.

[14] The Khronos Group. KHR_gaussian_splatting. https://github.com/KhronosGroup/glTF/blob/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md, 2026. Accessed 2026-07-13.

[15] The Khronos Group. Khronos Announces glTF Gaussian Splatting Extension. https://www.khronos.org/news/press/gltf-gaussian-splatting-press-release, 2026. Accessed 2026-07-13.

[16] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Quan Vuong, Philippe Hansen-Estruch, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Conference on Robot Learning*, 2023. Accessed 2026-07-13.